GuardianEye

Big Data Analytics (BDA) — Project Source Code

Course code 0701260704

Akhil Chivakula Dipti Kothari Khurinji Malar
Sukhada Dhingra

Department of Artificial Intelligence and Machine Learning
Symbiosis Institute of Technology

Guide: Dr. Archana Choudary
Head of the Department: Dr. Sumit Kumar

Academic Year 2026–2027

Abstract

This document contains the complete source of the Big Data Analytics coursework artefact `bda/guardianeye_bda_analysis.py`. The program opens a local PySpark session and aggregates prepared multimodal indicator metadata (image manifests, site-tagged audio, site-stratified splits, synthetic proxy pairs, and the SQLite audit log). It does not report detector precision, recall, F1, or mAP. One detection is an indicator, not proof of poaching.

Submission details

- Subject: Big Data Analytics (BDA)
- File path: `bda/guardianeye_bda_analysis.py`
- Run command: `.venv/bin/python bda/guardianeye_bda_analysis.py`
- Engine: PySpark 4.0.1, local[2], seed 27
- Group members:
  Akhil Chivakula — PRN 23070126009
  Dipti Kothari — PRN 23070126040
  Khurinji Malar — PRN 23070126059
  Sukhada Dhingra — PRN 24070126505

Source listing

`bda/guardianeye_bda_analysis.py`

```
"""Big Data Analytics coursework artefact for the GuardianEye indicator pipeline.

Distributed-style aggregation of multimodal indicator metadata: image-domain
manifests, site-tagged audio windows, site-stratified splits, synthetic proxy
fusion pairs, and the SQLite audit log. The job counts and groups those
records. It does not score a detector, and it is not a claim of field-validated
poaching detection. One detection is an indicator, not proof of poaching.

Primary engine: PySpark on a local master. Java and a Spark session are
```

```
required. Run from the repository root:

    .venv/bin/python bda/guardianeye_bda_analysis.py
"""

from __future__ import annotations

import csv
import json
import os
import random
import sqlite3
import subprocess
from pathlib import Path
from typing import Any

from pyspark.sql.functions import avg, col, to_timestamp, window
from pyspark.sql.functions import sum as spark_sum

SEED = 27

os.environ.setdefault("SPARK_LOCAL_IP", "127.0.0.1")

ROOT = Path(__file__).resolve().parents[1]
PROCESSED = ROOT / "data" / "processed"
EVENT_DB = ROOT / "data" / "events.sqlite"
REPORT_DIR = ROOT / "reports" / "bda"


def _java_version() -> str:
    proc = subprocess.run(["java", "-version"], capture_output=True, text=True, check=False)
    line = (proc.stderr or proc.stdout).splitlines()
    return line[0].strip() if line else "unknown"


def _ratio(counts: dict[str, int]) -> float | None:
    """Majority count divided by the smallest positive count."""
    positive = [count for count in counts.values() if count > 0]
    if len(positive) < 2:
        return None
    return max(positive) / min(positive)


def _as_dict(pairs: list[tuple[Any, int]]) -> dict[str, int]:
    out: dict[str, int] = {}
    for key, count in pairs:
        if isinstance(key, tuple):
            label = "|".join("" if part is None else str(part) for part in key)
        else:
            label = "" if key is None else str(key)
        out[label] = int(count)
    return dict(sorted(out.items()))


def _nested(pairs: list[tuple[Any, int]], depth: int) -> dict[str, Any]:
    """Fold (k1, k2, ...) reduce pairs into nested dicts."""
    root: dict[str, Any] = {}
    for key, count in pairs:
        if not isinstance(key, tuple):
            key = (key,)
        cursor = root
        for part in key[:-1]:
            part_s = "" if part is None else str(part)
            cursor = cursor.setdefault(part_s, {})
```

```
        leaf = "" if key[-1] is None else str(key[-1])
        cursor[leaf] = int(count)
    return root


def _load_jsonl_rdd(spark: Any, path: Path):
    if not path.is_file():
        return spark.sparkContext.emptyRDD()
    return spark.read.json(str(path)).rdd


def _image_jobs(spark: Any) -> dict[str, Any]:
    path = PROCESSED / "images" / "manifest.jsonl"
    rdd = _load_jsonl_rdd(spark, path)
    # Map each image record to grouping keys, then reduce by summation.
    total = rdd.count()
    by_domain = _as_dict(
        rdd.map(lambda row: (row["domain"], 1)).reduceByKey(lambda a, b: a + b).collect()
    )
    by_domain_class_pairs = (
        rdd.map(lambda row: ((row["domain"], row["primary_class"] or "empty"), 1))
        .reduceByKey(lambda a, b: a + b)
        .collect()
    )
    by_domain_split = _nested(
        rdd.map(lambda row: ((row["domain"], row["split"]), 1))
        .reduceByKey(lambda a, b: a + b)
        .collect(),
        2,
    )
    by_source = _as_dict(
        rdd.map(lambda row: (row["source"], 1)).reduceByKey(lambda a, b: a + b).collect()
    )
    empty_kept = rdd.filter(lambda row: (row["primary_class"] or "empty") == "empty").count()
    small_objects = rdd.filter(lambda row: bool(row["small_object"])).count()
    class_nested = _nested(by_domain_class_pairs, 2)
    ratios = {domain: _ratio(counts) for domain, counts in sorted(class_nested.items())}
    return {
        "records": int(total),
        "by_domain": by_domain,
        "by_domain_and_class": class_nested,
        "by_domain_and_split": by_domain_split,
        "by_source": by_source,
        "empty_images_retained": int(empty_kept),
        "small_object_images": int(small_objects),
        "imbalance_ratio_max_over_min": ratios,
    }


def _audio_jobs(spark: Any) -> dict[str, Any]:
    path = PROCESSED / "audio" / "manifest.jsonl"
    rdd = _load_jsonl_rdd(spark, path)
    total = rdd.count()
    by_site = _as_dict(
        rdd.map(lambda row: (row["site"], 1)).reduceByKey(lambda a, b: a + b).collect()
    )
    by_site_label_pairs = (
        rdd.map(lambda row: ((row["site"], row["label"]), 1))
        .reduceByKey(lambda a, b: a + b)
        .collect()
    )
    label_nested = _nested(by_site_label_pairs, 2)
    by_variant = _as_dict(
        rdd.map(lambda row: (row["variant"], 1)).reduceByKey(lambda a, b: a + b).collect()
```

```
    )
    by_representation = _as_dict(
        rdd.map(lambda row: (row["representation"], 1)).reduceByKey(lambda a, b: a + b).collect()
    )
    background_kept = rdd.filter(lambda row: row["label"] == "background").count()
    return {
        "records": int(total),
        "by_site": by_site,
        "by_site_and_label": label_nested,
        "by_variant": by_variant,
        "by_representation": by_representation,
        "background_clips_retained": int(background_kept),
        "imbalance_ratio_max_over_min": {
            site: _ratio(counts) for site, counts in sorted(label_nested.items())
        },
    }


def _split_jobs(spark: Any) -> dict[str, Any]:
    split_dir = PROCESSED / "audio" / "splits"
    files = sorted(split_dir.glob("*.jsonl")) if split_dir.is_dir() else []
    if not files:
        return {"experiments": {}, "files": 0}
    rdd = spark.read.json([str(path) for path in files]).rdd
    # Map (experiment, split, site, label) -> 1 and reduce.
    pairs = (
        rdd.map(lambda row: ((row["experiment"], row["split"], row["site"], row["label"]), 1))
        .reduceByKey(lambda a, b: a + b)
        .collect()
    )
    size_pairs = (
        rdd.map(lambda row: ((row["experiment"], row["split"]), 1))
        .reduceByKey(lambda a, b: a + b)
        .collect()
    )
    site_pairs = (
        rdd.map(lambda row: ((row["experiment"], row["split"], row["site"]), 1))
        .reduceByKey(lambda a, b: a + b)
        .collect()
    )
    return {
        "files": len(files),
        "records": int(rdd.count()),
        "by_experiment_and_split": _nested(size_pairs, 2),
        "by_experiment_split_site": _nested(site_pairs, 3),
        "by_experiment_split_site_label": _nested(pairs, 4),
    }


def _proxy_jobs(spark: Any) -> dict[str, Any]:
    path = PROCESSED / "paired" / "events.jsonl"
    rdd = _load_jsonl_rdd(spark, path)
    total = rdd.count()
    by_protocol = _as_dict(
        rdd.map(lambda row: (row["protocol"], 1)).reduceByKey(lambda a, b: a + b).collect()
    )
    by_scenario = _as_dict(
        rdd.map(lambda row: (row["scenario"], 1)).reduceByKey(lambda a, b: a + b).collect()
    )
    by_level = _as_dict(
        rdd.map(lambda row: (row["expected_level"], 1)).reduceByKey(lambda a, b: a + b).collect()
    )
    by_domain = _as_dict(
        rdd.map(lambda row: (row["domain"], 1)).reduceByKey(lambda a, b: a + b).collect()
```

```
    )
    return {
        "records": int(total),
        "protocol": "synthetic_proxy",
        "note": "Constructed pairs. Not time-aligned field incidents.",
        "by_protocol": by_protocol,
        "by_scenario": by_scenario,
        "by_expected_level": by_level,
        "by_domain": by_domain,
    }


def _event_rows() -> list[dict[str, Any]]:
    if not EVENT_DB.is_file():
        return []
    conn = sqlite3.connect(EVENT_DB)
    conn.row_factory = sqlite3.Row
    try:
        tables = {
            row[0] for row in conn.execute("SELECT name FROM sqlite_master WHERE type='table'")
        }
        if "events" not in tables:
            return []
        rows = []
        for record in conn.execute("SELECT event_id, timestamp, alerted, payload FROM events"):
            payload = json.loads(record["payload"]) if record["payload"] else {}
            rows.append(
                {
                    "event_id": str(record["event_id"]),
                    "timestamp": str(record["timestamp"]),
                    "alerted": int(record["alerted"] or 0),
                    "risk_level": str(payload.get("risk_level") or ""),
                    "domain": str(payload.get("domain") or ""),
                    "reason": str(payload.get("reason") or ""),
                    "latency_ms": float(payload.get("latency_ms") or 0.0),
                    "modalities": ",".join(payload.get("modalities_present") or []),
                }
            )
        return rows
    finally:
        conn.close()


def _event_jobs(spark: Any) -> dict[str, Any]:
    rows = _event_rows()
    if not rows:
        return {"records": 0, "windows_1h": [], "by_risk_level": {}, "by_reason": {}}
    # Map log rows into a frame, then reduce inside tumbling windows.
    frame = spark.createDataFrame(rows).withColumn("ts", to_timestamp(col("timestamp")))
    windowed = (
        frame.groupBy(window(col("ts"), "1 hour").alias("bucket"))
        .agg(
            spark_sum(col("alerted")).alias("alerted"),
            spark_sum((col("alerted") == 0).cast("int")).alias("not_alerted"),
            avg(col("latency_ms")).alias("mean_latency_ms"),
        )
        .orderBy("bucket")
        .collect()
    )
    windows = []
    for row in windowed:
        bucket = row["bucket"]
        windows.append(
            {
```

```
                "start": bucket["start"].isoformat(sep=" ") if bucket and bucket["start"] else None,
                "end": bucket["end"].isoformat(sep=" ") if bucket and bucket["end"] else None,
                "alerted": int(row["alerted"] or 0),
                "not_alerted": int(row["not_alerted"] or 0),
                "mean_latency_ms": round(float(row["mean_latency_ms"] or 0.0), 3),
            }
        )
    rdd = frame.rdd
    by_level = _as_dict(
        rdd.map(lambda row: (row["risk_level"], 1)).reduceByKey(lambda a, b: a + b).collect()
    )
    by_reason = _as_dict(
        rdd.map(lambda row: (row["reason"], 1)).reduceByKey(lambda a, b: a + b).collect()
    )
    by_domain = _as_dict(
        rdd.map(lambda row: (row["domain"], 1)).reduceByKey(lambda a, b: a + b).collect()
    )
    alerted = int(rdd.map(lambda row: int(row["alerted"])).reduce(lambda a, b: a + b))
    return {
        "records": len(rows),
        "alerted": alerted,
        "not_alerted": len(rows) - alerted,
        "by_risk_level": by_level,
        "by_reason": by_reason,
        "by_domain": by_domain,
        "windows_1h": windows,
    }


def _flatten(section: str, payload: Any, prefix: str = "") -> list[tuple[str, str, str]]:
    rows: list[tuple[str, str, str]] = []
    if isinstance(payload, dict):
        for key, value in payload.items():
            child = f"{prefix}.{key}" if prefix else str(key)
            rows.extend(_flatten(section, value, child))
        return rows
    if isinstance(payload, list):
        for index, value in enumerate(payload):
            child = f"{prefix}[{index}]"
            rows.extend(_flatten(section, value, child))
        return rows
    rows.append((section, prefix or section, "" if payload is None else str(payload)))
    return rows


def _write_outputs(summary: dict[str, Any]) -> tuple[Path, Path]:
    REPORT_DIR.mkdir(parents=True, exist_ok=True)
    json_path = REPORT_DIR / "summary.json"
    csv_path = REPORT_DIR / "aggregation.csv"
    json_path.write_text(json.dumps(summary, indent=2, sort_keys=True) + "\n", encoding="utf-8")
    flat: list[tuple[str, str, str]] = []
    for section, payload in summary.items():
        if section in {"engine", "seed", "note"}:
            flat.append(
                (
                    section,
                    section,
                    str(payload) if not isinstance(payload, dict) else json.dumps(payload),
                )
            )
            if isinstance(payload, dict):
                flat.extend(_flatten(section, payload))
            continue
        flat.extend(_flatten(section, payload))
```

```
    with csv_path.open("w", encoding="utf-8", newline="") as handle:
        writer = csv.writer(handle)
        writer.writerow(["section", "key", "value"])
        writer.writerows(flat)
    return json_path, csv_path


def _print_log(summary: dict[str, Any], json_path: Path, csv_path: Path) -> None:
    image = summary["image"]
    audio = summary["audio"]
    splits = summary["audio_splits"]
    proxy = summary["synthetic_proxy_pairs"]
    events = summary["event_log"]
    lines = [
        "===== BDA RUN LOG =====",
        f"engine: {summary['engine']['name']} {summary['engine']['version']}",
        f"java: {summary['engine']['java']}",
        f"master: {summary['engine']['master']}",
        f"seed: {summary['seed']}",
        f"image_records: {image['records']}",
        f"image_by_domain: {json.dumps(image['by_domain'], sort_keys=True)}",
        f"image_by_domain_and_class: {json.dumps(image['by_domain_and_class'], sort_keys=True)}",
        f"image_by_domain_and_split: {json.dumps(image['by_domain_and_split'], sort_keys=True)}",
        f"image_by_source: {json.dumps(image['by_source'], sort_keys=True)}",
        f"empty_images_retained: {image['empty_images_retained']}",
        f"small_object_images: {image['small_object_images']}",
        f"image_imbalance_ratio: {json.dumps(image['imbalance_ratio_max_over_min'], sort_keys=True)}",
        f"audio_records: {audio['records']}",
        f"audio_by_site: {json.dumps(audio['by_site'], sort_keys=True)}",
        f"audio_by_site_and_label: {json.dumps(audio['by_site_and_label'], sort_keys=True)}",
        f"audio_by_variant: {json.dumps(audio['by_variant'], sort_keys=True)}",
        f"audio_by_representation: {json.dumps(audio['by_representation'], sort_keys=True)}",
        f"background_clips_retained: {audio['background_clips_retained']}",
        f"audio_imbalance_ratio: {json.dumps(audio['imbalance_ratio_max_over_min'], sort_keys=True)}",
        f"audio_split_records: {splits.get('records', 0)}",
        f"audio_splits: {json.dumps(splits.get('by_experiment_and_split', {}), sort_keys=True)}",
        f"audio_split_sites: {json.dumps(splits.get('by_experiment_split_site', {}), sort_keys=True)}",
        f"proxy_records: {proxy['records']}",
        f"proxy_by_protocol: {json.dumps(proxy['by_protocol'], sort_keys=True)}",
        f"proxy_by_scenario: {json.dumps(proxy['by_scenario'], sort_keys=True)}",
        f"proxy_by_expected_level: {json.dumps(proxy['by_expected_level'], sort_keys=True)}",
        f"proxy_by_domain: {json.dumps(proxy['by_domain'], sort_keys=True)}",
        f"event_log_records: {events['records']}",
        f"event_log_alerted: {events.get('alerted', 0)}",
        f"event_log_not_alerted: {events.get('not_alerted', 0)}",
        f"event_log_by_risk_level: {json.dumps(events.get('by_risk_level', {}), sort_keys=True)}",
        f"event_log_by_reason: {json.dumps(events.get('by_reason', {}), sort_keys=True)}",
        f"event_log_by_domain: {json.dumps(events.get('by_domain', {}), sort_keys=True)}",
        f"event_log_windows_1h: {json.dumps(events.get('windows_1h', []), sort_keys=True)}",
        f"wrote: {json_path}",
        f"wrote: {csv_path}",
        "===== END BDA RUN LOG =====",
    ]
    print("\n".join(lines))


def main() -> None:
    random.seed(SEED)
    from pyspark.sql import SparkSession

    spark = (
        SparkSession.builder.master("local[2]")
        .appName("guardianeye-bda")
        .config("spark.ui.enabled", "false")
```

```
        .config("spark.driver.host", "127.0.0.1")
        .config("spark.sql.shuffle.partitions", "2")
        .getOrCreate()
    )
    spark.sparkContext.setLogLevel("ERROR")
    try:
        summary: dict[str, Any] = {
            "seed": SEED,
            "note": (
                "Analytical summary of prepared indicator metadata. "
                "Not model metrics. Fusion pairs use the synthetic proxy protocol."
            ),
            "engine": {
                "name": "pyspark",
                "version": spark.version,
                "master": "local[2]",
                "java": _java_version(),
            },
            "image": _image_jobs(spark),
            "audio": _audio_jobs(spark),
            "audio_splits": _split_jobs(spark),
            "synthetic_proxy_pairs": _proxy_jobs(spark),
            "event_log": _event_jobs(spark),
        }
        json_path, csv_path = _write_outputs(summary)
        _print_log(summary, json_path, csv_path)
    finally:
        spark.stop()


if __name__ == "__main__":
    main()
```